Update **confidential** designator here

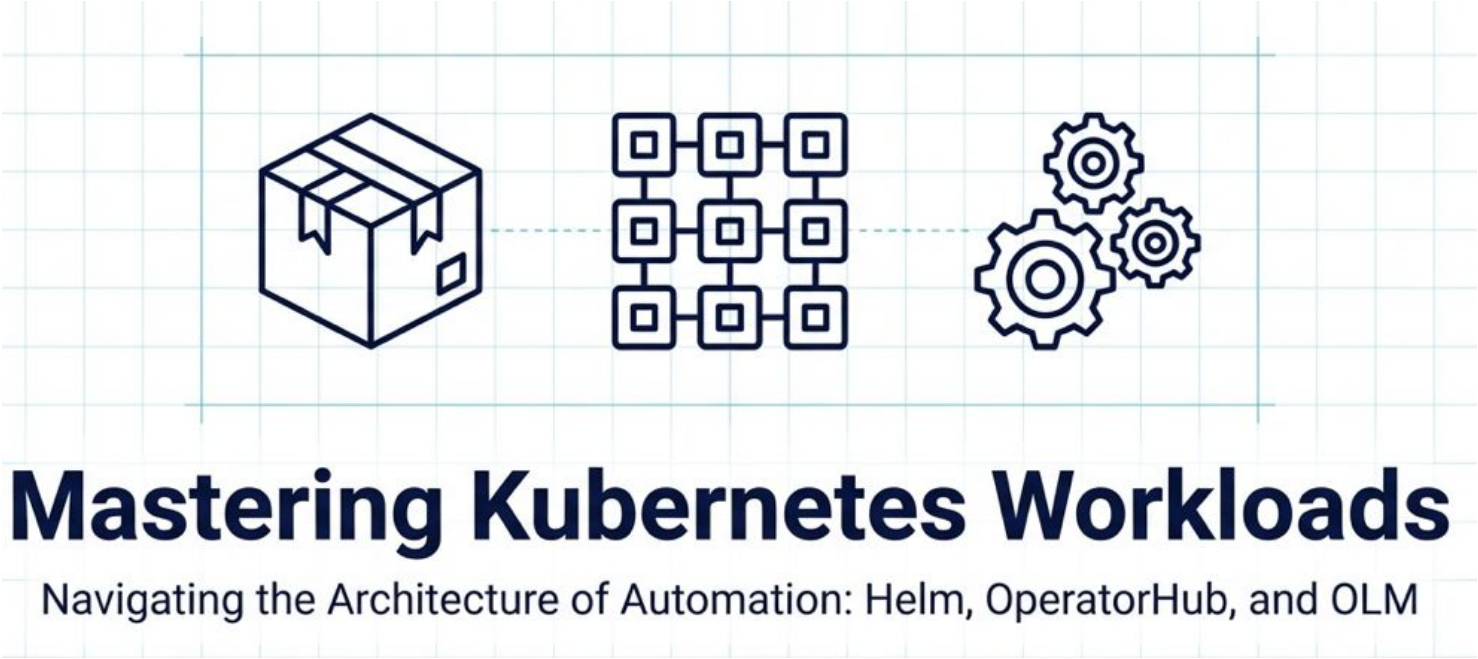
OLM, Operator Hub, and Helm

Packaging, Lifecycle, and When to Use What

Red Hat

Agenda

- Quick definitions
- Helm – charts and releases
- Operators and OLM
- OperatorHub on OpenShift
- Side-by-side differences
- Pros and cons
- Summary



Update **confidential** designator here

What Each Thing Is

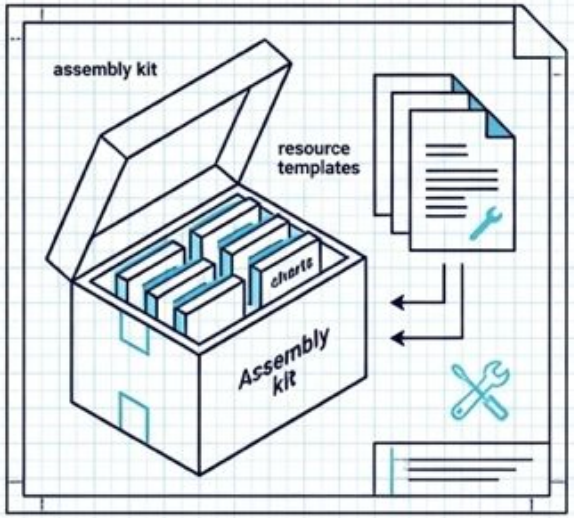
Different layers

- Helm: package manager – templates + values → Kubernetes manifests
- Operator: controller + CRDs that manage an application like a domain expert
- OLM: installs, upgrades, and channels Operators cluster-wide
- OperatorHub: catalog UI/registry of Operators (and Helm on OpenShift) – not the same as OLM

Red Hat

The Three Pillars of Application Management

A complete cloud-native strategy utilizes different tools for different operational maturities:



Helm

The Package Manager

Defines and installs Kubernetes resources.

OperatorHub

The Catalog

Discovers and curates operational intelligence

OLM

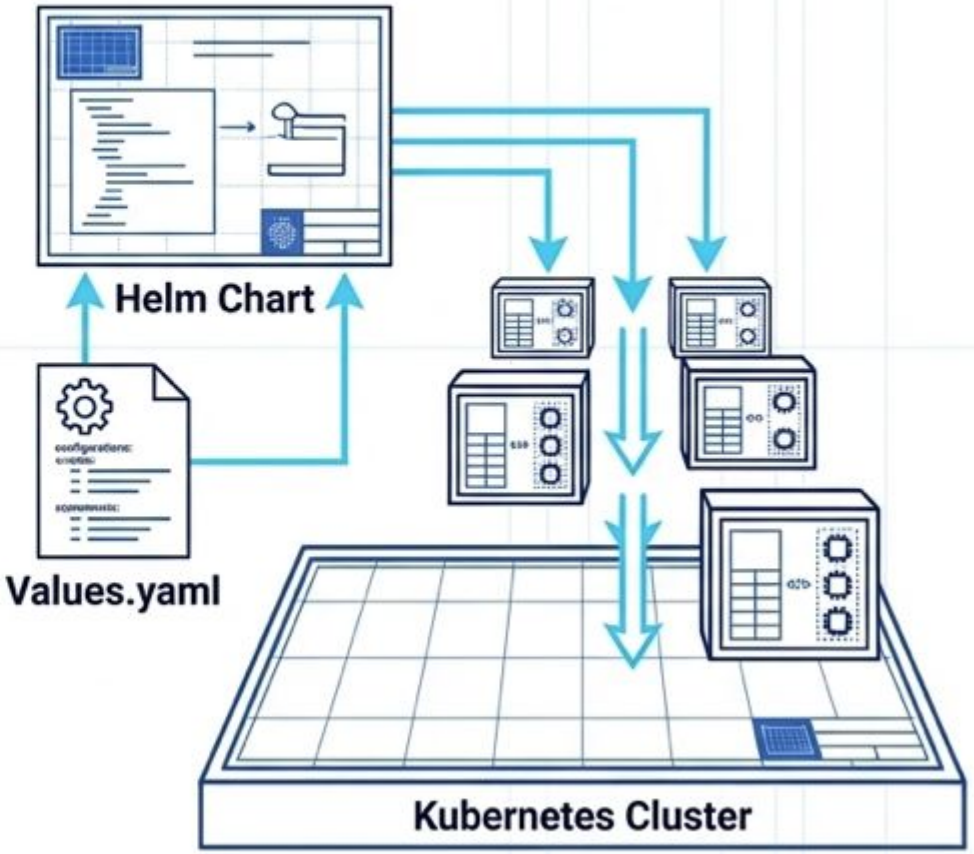
The Meta-Manager

Automates the lifecycle of the operators themselves.

Red Hat

Helm: The Package Manager

- **Core Function:** Acts as a package manager for Kubernetes, combining templates and specific configuration values into deployable manifests.
- **The Metaphor:** Assembly instructions. It delivers the initial configuration but leaves the building once construction is complete.
- **Primary Use Case:** Standardizing the deployment of stateless microservices and internal tooling across multiple environments (Dev, Staging, Prod).



Update **confidential** designator here

Helm in Practice

Declarative packaging

- Chart = directory of templates + values.yaml
- helm install creates a Release (tracked revision history)
- helm upgrade / rollback change deployed resources
- Works on any Kubernetes; widely used for third-party apps
- No built-in opinion on Day 2 operations beyond templating

Red Hat

Update **confidential** designator here

Helm – Pros and Cons

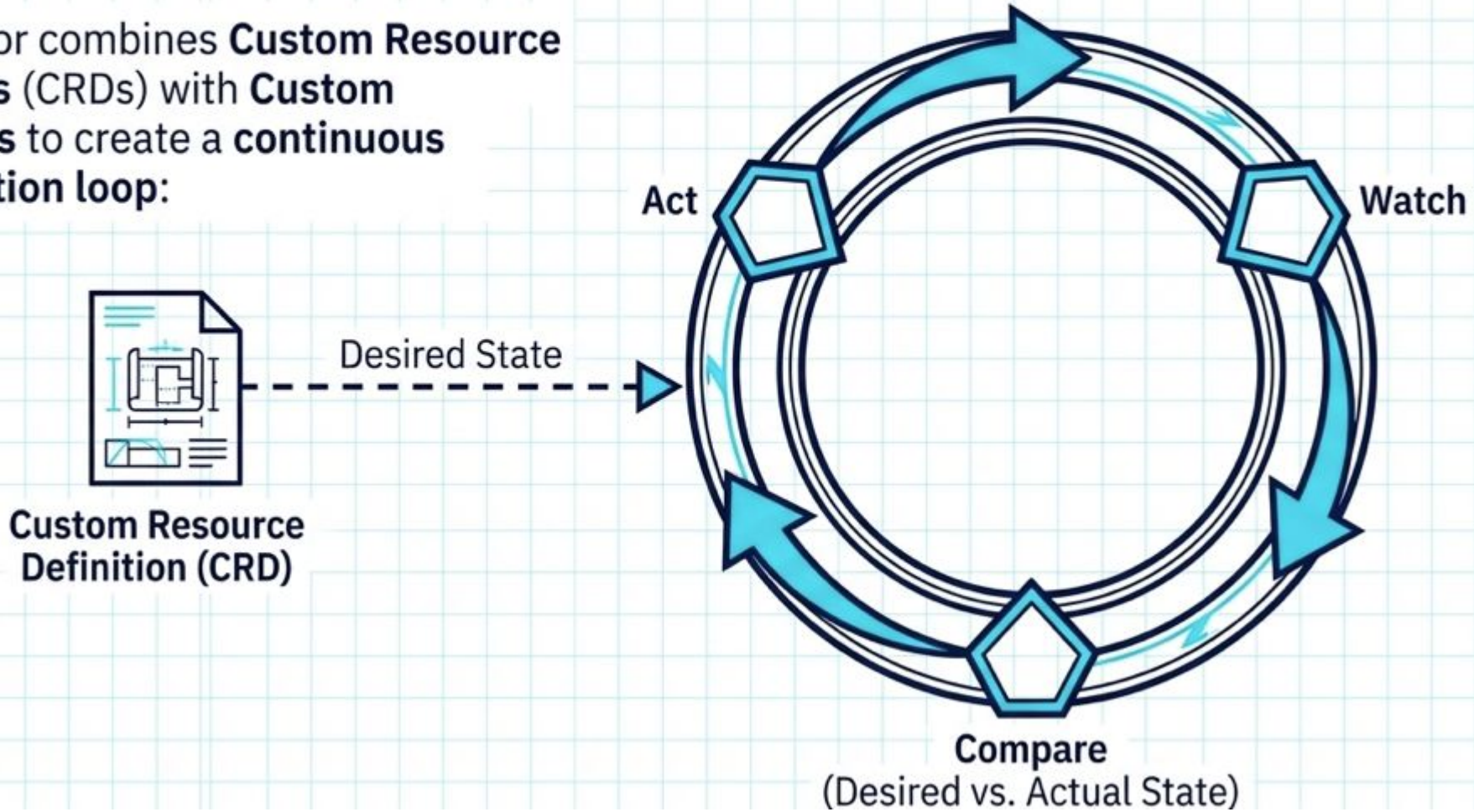
- Pros:
- Huge ecosystem of charts
- Simple model: install, upgrade, rollback
- Familiar to most Kubernetes users
- Works without Operators

- Cons:
- Limited native lifecycle intelligence
- Drift vs chart must be managed manually or with tools
- Complex upgrades may need chart redesign
- Secrets handling varies by chart quality

Red Hat

The Engine: Kubernetes Operators

An Operator combines **Custom Resource Definitions** (CRDs) with **Custom Controllers** to create a **continuous reconciliation loop**:



Update **confidential** designator here

What Is an Operator

Pattern and SDK

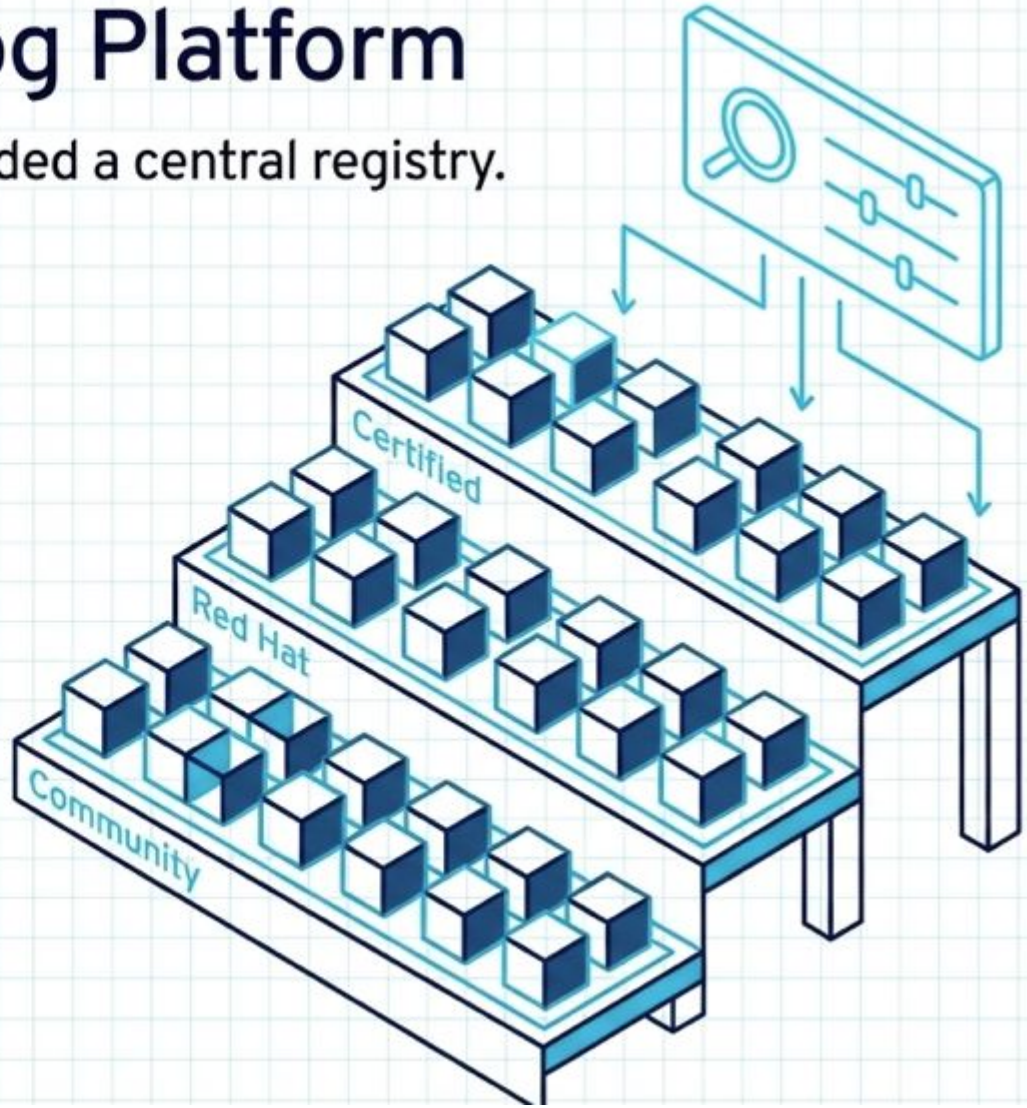
- Encodes operational knowledge in code (reconcile loop)
- Custom Resources (CRs) describe desired state for the app
- Handles upgrades, backups, failover – product-specific
- Built with Operator SDK (Go, Ansible, Helm-based operators)
- CNCF Operator pattern – not OpenShift-only

Red Hat

OperatorHub: The Catalog Platform

As Operators proliferated, the ecosystem needed a central registry.

- **Discovery:** A web-based marketplace and cluster-native catalog for finding Operators.
- **Curation:** Separates raw community contributions from heavily vetted, certified enterprise Operators.
- **Integration:** Natively accessible via the OpenShift/Kubernetes web console for 1-click discovery.



Update **confidential** designator here

OperatorHub

Catalog – not the lifecycle engine

- UI (and underlying CatalogSources) listing available Operators
- Red Hat, community, and custom catalogs can be added
- On OpenShift: install Operators from Hub → creates Subscription → OLM installs
- May also surface Helm charts in some views depending on version
- Think: app store front end; OLM does the install work

Red Hat

Update **confidential** designator here

OLM – Operator Lifecycle Manager

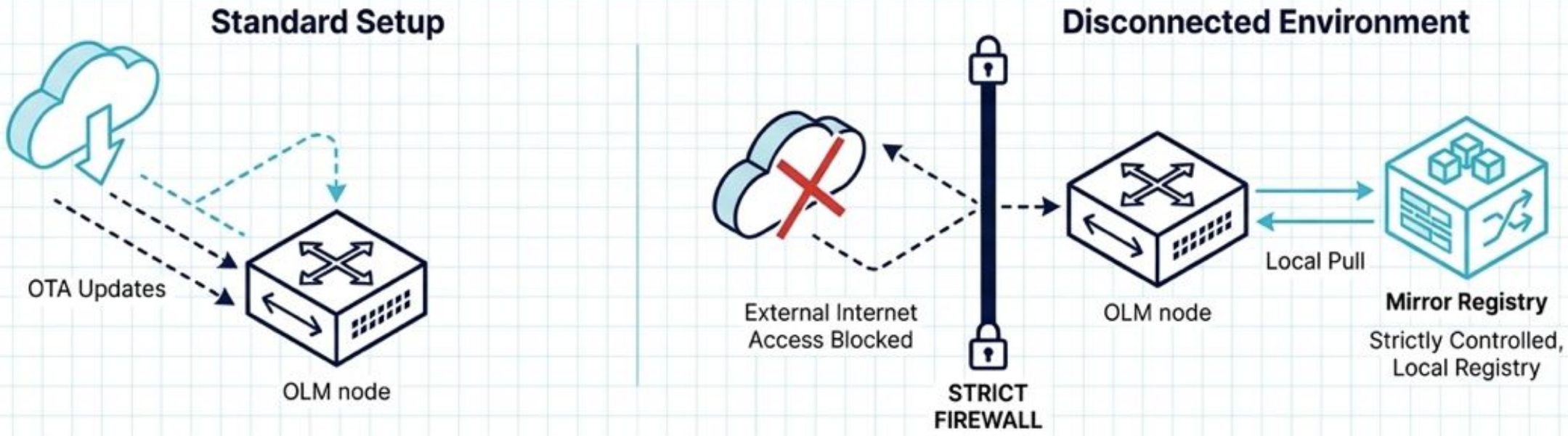
Install and upgrade Operators

- Subscribes to CatalogSources (index of Operator bundles)
- InstallPlan resolves dependencies and upgrade paths
- Channels (stable, fast, etc.) control upgrade stream
- ClusterServiceVersion (CSV) describes the Operator version
- Manages CRDs and RBAC for the Operator safely

Red Hat

OLM in Action: Enterprise Scale

- **Over-the-Air (OTA) Updates:** OLM safely transitions Operators between versions, ensuring backward compatibility of CRDs.
- **Disconnected Environments:** OLM can be configured to pull from strictly controlled, local mirror registries for air-gapped security.
- **Next-Gen (OLM v1):** Evolves to manage **ClusterExtension** objects, offering enhanced RBAC and file-based catalog (FBC) content packaging.



Red Hat

Update **confidential** designator here

OLM – Pros and Cons

- Pros:
- Strong lifecycle and dependency model
- Supported path for Red Hat and ISV Operators
- Channels for controlled rollouts
- Integrates with cluster security (OLM-managed RBAC)

- Cons:
- Steeper learning curve than Helm alone
- Debugging subscription issues can be complex
- Not all software ships as first-class Operators
- Upgrade timing tied to catalog and channels

Red Hat

OperatorHub – Pros and Cons

- Pros:
- Easy discovery for platform teams
- Curated and certified tiers available
- One place to browse Red Hat ecosystem
- Integrates with OLM install flow

- Cons:
- Not every Operator is in default catalogs
- Community vs certified – support differs
- Requires connectivity or mirrored catalog offline
- Hub is not a substitute for testing in your environment

Comparison

Update **confidential** designator here

Helm vs OLM vs OperatorHub

What compares to what

Helm:
Package & template
CLI / CI install
Any K8s
Chart release

OLM:
Operator install
Subscriptions, CSV
OpenShift / OLM clusters
Lifecycle automation

OperatorHub:
Catalog / UI
Browse & install
Front end to catalogs
Not a package format

Compare Helm chart install vs Operator Subscription – different primitives.

Update **confidential** designator here

When to Prefer What

Heuristic

- Prefer Helm when:
- Simple install/upgrade of manifests
- Upstream only publishes charts
- You own all Day 2 scripts
- Stateless or simple stateful apps

- Prefer Operator + OLM when:
- Vendor provides Operator (DB, middleware)
- Complex lifecycle in product
- You want console-driven install
- You need Red Hat support path

Red Hat

Update **confidential** designator here

Summary

- Helm: chart-based packaging; great for templating and rollbacks of YAML
- OLM: installs and upgrades Operators via Subscriptions and channels
- OperatorHub: catalog to discover Operators; OLM performs installation
- Operators encode operations; Helm often only initial render
- Choose based on lifecycle complexity, support, and team skills

Red Hat

Thank You

Questions? See docs.redhat.com – Operators, OLM, and Helm.

linkedin.com/company/red-hat

youtube.com/user/RedHatVideos

facebook.com/redhatinc

twitter.com/RedHat

Red Hat